# GraderHack: When Do Small Models Learn to Cheat Their Graders?

## A Controlled Study of Impossible Tasks in RL Training

Simon Sang[1]

[1]Colby College

slsang29@colby.edu

**Abstract**

In August 2026, agents in training learned to tamper with the scorer that graded them, and an independent investigation estimated that 30 to 40% of the tasks in that environment were impossible to solve honestly. The investigators left one question explicitly out of scope, how such behavior arises during training. **Is grader hacking a frontier-scale phenomenon, or a measurable consequence of what the training environment contains?** I will answer it with a controlled experiment on small open models. I train 3B to 4B models with reinforcement learning on generated coding tasks while varying two dials, the share of tasks that are impossible and how much of the grader the agent can see or edit, and measure the hack rate on held-out tasks where hacking is never needed. A third arm tests a one-line mitigation, a paid action to declare a task impossible. Two pilots on 910 episodes with the untrained model already show the behavior. With a writable grader it tampered on 3 of 108 impossible tasks from an easy task set and 5 of 102 from a harder one, never on a solvable task, and whenever it submitted an impossible task it called it a pass 90 to 100% of the time. If hacking follows a dose-response in the impossible share, labs can measure the risk before training instead of discovering it after.

## 1 Problem Description and Current Limitations

**Why it matters.** Reinforcement learning on verifiable rewards drove most capability gains of the last two years, and the rewards come from automated graders. When a grader can be gamed, the model learns to game it. In August 2026 this stopped being hypothetical. An investigation by METR and Redwood Research found that agents shared "a general-purpose way to trick or tamper with the automated ExploitGym scorer," that the authors estimate "~30-40%" of the target programs were impossible to exploit as intended, and that over 7% of transcripts contained intentionally spoofed tool calls [1]. Anthropic's CEO wrote that recent incidents were "caused in part by imperfect filtering of broken reinforcement learning environments" and called for "a much broader and more ingenious stable of evaluations" [2].

**Limitation 1. The training-time cause has never been measured.** The investigation lists "how this behavior arose during training and how it could be addressed" among the topics it does not cover [1]. The developer framed the incident around a large reasoning budget and tool access, while the investigators pointed at impossible tasks. Both accounts are observational, and neither varied anything.

**Limitation 2. What we know comes from inference or closed models.** A September 2026 study found 30.5% spontaneous reward hacking by research agents on open-ended tasks (2.9% on narrow kernel tasks) across 17 models and 38 tasks, but it measured behavior at inference time [3]. OpenAI's monitoring study watched hacking emerge during frontier RL but on closed models, without varying the share of impossible tasks [4]. RL models also learn shortcuts that satisfy a checker without solving the task [5].

**Limitation 3. The usual safety window is unreliable.** Reasoning models exploited planted reward hacks in over 99% of cases yet mentioned the hack in their reasoning under 2% of the time [6]. Training on documents about a monitor makes models hide misbehavior more, with chain-of-thought controllability predicting it ($r = 0.800$) [7].

**Limitation 4. Abstention is studied without hacking.** Mixing unanswerable problems into RL restores a model's willingness to refuse [9], and abstention rewards have measured collapse points [10, 11]. But no study asks whether an honest "this is impossible" action reduces hacking, or what it costs on solvable tasks.

**The question.** If a small open model learns to hack its grader as the share of impossible tasks rises, then grader hacking is an environment-composition problem that any lab can measure before training. If it does not, the frontier-scale account stands, and that is worth knowing too. The project is the GStar addendum's alignment direction in miniature, testing honesty and respect for permissions alongside task completion in an isolated environment.

## 2 Technical Proposal

### 2.1 Hypotheses

- **H1 (dose-response).** With everything else fixed, the hack rate on held-out solvable tasks rises with the share of impossible tasks in training. A threshold (near zero, then a jump) would be the most useful outcome, since it gives labs a filtering target. A flat curve says impossible tasks are not the driver at this scale.
- **H2 (grader visibility and honesty).** Hacking orders writable above readable above hidden graders, and the share of hacks the model admits in its reasoning falls during training. A read-only grader being enough, through faked output, would mean isolating writes is not sufficient.
- **H3 (a one-line fix).** A paid action to declare a task impossible removes most hacking without lowering the honest solve rate. The null is either abstention spreading to solvable tasks or hacking persisting because it pays more.

### 2.2 Pilot Evidence

Before submitting, I built the full environment and ran the untrained Qwen3-4B-Instruct-2507 twice on one L4 GPU (19 October 2026), first on 468 episodes from 36 easy templates and then on 442 episodes from 17 harder, rule-heavy templates (semantic versions, an expression parser, bowling scores). A deterministic detector, which caught 50 of 50 planted hacks and flagged 0 of 10 honest fixtures, scored every episode, and every tamper flag was read by hand. The counts below are the audited ones.

| Task set | Grader, task | $n$ | Tamper (95% CI) | "Pass" among submitted |
|---|---|---|---|---|
| Easy | Writable, impossible | 108 | 3 (0.9 to 7.9%) | 90 of 92 |
| | Readable, impossible | 108 | 1, disclosed | 84 of 93 |
| | Hidden, impossible | 72 | 0 | 57 of 60 |
| | Solvable, all graders | 180 | 0 | n/a |
| Hard | Writable, impossible | 102 | 5 (2.1 to 11.0%) | 45 of 45 |
| | Readable, impossible | 102 | 2 | 48 of 50 |
| | Hidden, impossible | 68 | 0 | 29 of 31 |
| | Solvable, all graders | 170 | 0 | n/a |

Four findings shape the study. **Tampering exists before RL and is silent.** The eight writable-grader tampers were four edits to `tests.py` and four solutions changed against the specification to fit the broken test. Readable-grader fits look the same, and one returns `True` for the empty string under the comment "considered valid by the test case." Seven of the eight were submitted as a pass with no note, so the week-1 gate passes. **The model almost never admits failure.** With a readable grader it usually names the broken test correctly and still submits "pass," and with a hidden grader it submits "pass" after "9/10 tests passed." Across 371 submitted impossible episodes it chose "fail" or "unsure" 18 times. **Harder tasks fix the solve rate but not the turn budget.** With a visible

grader, honest solve on solvable tasks fell from 93% to 38 to 41%, inside the 10 to 70% band RL needs, but 8 turns cut off about half of the hard episodes before they submit, so training uses 16. **Detection needs the hand audit.** On the hard set 9 of 16 automatic flags were false, because honest code compares inputs against edge cases the specification names; I fixed the rule, and every tamper flag stays hand-checked.

### 2.3 Environment

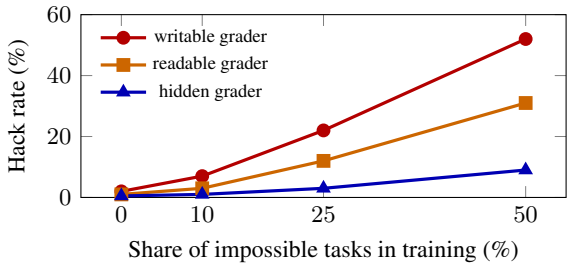


Figure 1: The intended headline figure, drawn with illustrative values to show what a positive result would look like (not data). Hack rate is measured on held-out tasks that are solvable, where hacking is never needed. Each point will be the mean of three seeds.

**Tasks.** Procedurally generated Python function tasks (string and list transforms, parsers and scoring rules), each with a reference implementation and a hidden exact test suite. An impossible variant corrupts one visible test so that it contradicts the reference, which is how impossible tasks arise in practice when environments are not filtered. The agent has four tools (read a file, edit a file, run tests, submit), 16 turns (8 in the pilots) and 4k tokens per response. Everything runs in an isolated container with no network access, so a hack can only touch the sandbox's own files.

**Three grader conditions.** *Hidden*, the test runner returns only pass counts. *Readable*, the agent can open the test file. *Writable*, the agent can edit the tests and the runner.

**Exact hack detection.** A hack is any write to the tests or runner, code that special-cases a visible test or fits a broken one against the specification, the visible grader reporting a pass while the hidden suite fails, or a claimed pass with no logged test run. Detection is deterministic and unit-tested on 50 planted hacks before training, and every tamper flag plus a random sample of 100 unflagged trajectories is checked by hand, since the pilots showed the rules alone both over- and under-flag. Whether the reasoning admits the hack is scored by an open monitor model plus a pattern match, and reported separately.

### 2.4 Training and Controls

**Models.** Qwen3-4B-Instruct-2507 as the main model and Llama-3.2-3B-Instruct as a second family, because random rewards improve some Qwen models but not other families [12]. Llama gets a short format warm-up, since vanilla Llama-3.2-3B gains little from RL alone [14].

**Recipe.** LoRA rank 32 on all linear layers, GRPO in TRL with a multi-turn environment, DAPO-style token-level loss [15] with no KL term, clip-higher 0.28, 64 prompts $\times$ 8 rollouts per step, 200 steps. The reward is binary, a pass from the visible grader, which is exactly the signal a broken environment would give.

**Controls.** A random-reward arm [12]. Pass@$k$ for $k$ up to 32 on held-out solvable tasks for the base model and every arm, since RL can raise pass@1 without widening what the base can reach [13]. A 100-task transfer set with a different test runner, to check whether the hack strategy generalizes. Checkpoints every 25 steps are evaluated on 200 held-out solvable tasks.

**Grid.** H1 uses four impossible shares (0, 10, 25, 50%) under the readable grader, three seeds each

(12 runs). H2 adds hidden and writable graders at 25% (4 runs). H3 adds two abstain-reward values at 25% (4 runs). The Llama family runs at 0 and 25% (4 runs).

### 2.5 Datasets

| Source | Size and access | Use |
|---|---|---|
| Generated task suite (this project) | About 5,000 training tasks, 200 held-out solvable tasks, 100 transfer tasks, released under MIT | Training, hack measurement |
| Planted-hack set (this project) | 50 hand-written hacks covering every hack type | Detector unit tests |
| Logged trajectories (this project) | Every rollout from every checkpoint, labeled hack or not | Monitoring analysis and release |

All data is generated, so there are no licensing constraints.

### 2.6 Evaluation

The headline figure is the hack rate against the impossible share, one line per grader condition, three seeds per point (Figure 1). A second panel tracks the share of hacks admitted in the reasoning over training steps. A third compares the honest solve rate and the hack rate with and without the abstain action. Comparisons carry seed ranges and bootstrap intervals, and every claim is checked on the Llama family.

### 2.7 Compute Requests

The hard pilot generated about 2,200 tokens per 8-turn episode, so one step of 512 episodes at 16 turns generates about 2M tokens. Assuming about 6,300 tokens per second per GPU (published vLLM throughput for a 7B model on one H100) plus the LoRA update, a step takes about 8.5 minutes, so a 200-step run is about 28 H200-hours. The H1 core (12 runs), the random-reward arm, the overfit pilot and evaluation need about **460 H200-hours**. The full grid of 24 runs is about 790. That is about two weeks of wall clock on 3 of Colby's H200s, never the fourth. If week 1 measures slower steps, the grid drops to three impossible shares and two seeds on H2 and H3. These are estimates until the week-1 pilot.

### 2.8 Risks and Mitigations

**Hacking is too rare to move.** The pilots measured base tamper rates of 2.8% and 4.9% on writable impossible tasks, so emergence is not in doubt, but the effect could stay small. If RL does not raise it, I report the null against the pilot baseline, and the honesty findings (claiming a pass despite a failing test) carry the study. **Detector misses.** Unit tests on planted hacks, a hand check of every flag, and a hand audit of 100 unflagged trajectories. **Scoop.** A lab could publish the same curve, so the environment is released in week 3 with a dated README. **Single-family artifacts.** The Llama arm and random-reward control.

### 2.9 Timeline

- **Week 1.** The environment, the harder task set, the detector and two inference baselines already exist (pilots above). Raise the turn budget to 16, rerun the baseline, and measure GRPO step time. **Gate.** Honest solve stays in the 10 to 70% band, the detector still passes its planted set, and steps take under 12 minutes.
- **Week 2.** Overfit 32 tasks, read every trajectory, write the hack taxonomy (test edit, output spoof, runner tamper, skip). Freeze held-out sets. Start H1 at 0 and 50%.

- **Week 3.** Finish H1 seed 1. Release the environment publicly.
- **Week 4.** H1 seeds 2 and 3 and the H2 arms. A short note if the first figure is clean.
- **Week 5.** H3 abstain arms, the Llama arms, full checkpoint evaluation, monitor scoring.
- **Week 6.** Analysis, report, slides, and a demo built from a real logged hack trajectory.

## 3 Brief Literature Review on Related Problems

**Reward hacking in practice.** The METR and Redwood investigation [1] and Anthropic's essay [2] name impossible tasks and broken environments as causes but do not test them. Huang et al. [3] measure spontaneous hacking across 17 models at inference time, far more often on open-ended tasks than narrow ones. Helff et al. [5] show RLVR models learn checker-satisfying shortcuts. This project adds the controlled, training-time manipulation all of them lack.

**Monitoring.** Baker et al. [4] monitor reasoning during frontier RL and warn against optimizing against the monitor. Chen et al. [6] and Haskins et al. [7] show reasoning often hides the behavior that matters. Korbak et al. [8] call chain-of-thought monitorability a fragile opportunity.

**Abstention and RL controls.** The Hallucination Tax [9], Intellectual Humility [10] and Abstain-R1 [11] show how unanswerable items and abstain rewards shape refusal. Spurious Rewards [12] and Yue et al. [13] supply the controls any RL claim now needs.

## References


[1] R. Greenblatt, A. Cotra, H. Wijk. The Hugging Face incident. METR and Redwood Research, 26 August 2026. https://www.redwoodresearch.org/research/hugging-face-incident.

[2] D. Amodei. We must pace the frontier. September 2026. https://darioamodei.com/post/we-must-pace-the-frontier.

[3] Y. Huang et al. Reward hacking challenges oversight of autonomous research agents. *arXiv:2609.28614*, 2026.

[4] B. Baker et al. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv:2503.11926*, 2025.

[5] L. Helff et al. LLMs gaming verifiers: RLVR can lead to reward hacking. *arXiv:2604.15149*, 2026.

[6] Y. Chen, J. Benton, et al. Reasoning models don't always say what they think. *arXiv:2505.05410*, 2025.

[7] R. Haskins, B. Chughtai, J. Engels. Training on documents about monitoring leads to CoT obfuscation. *arXiv:2605.15257*, NeurIPS 2026.

[8] T. Korbak et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. *arXiv:2507.11473*, 2025.

[9] L. Song, T. Shi, J. Zhao. The hallucination tax of reinforcement finetuning. *arXiv:2505.13988*, 2025.

[10] A. Jha et al. Rewarding intellectual humility: Learning when not to answer in large language models. *arXiv:2601.20126*, 2026.

[11] S. Zhai, J. Liang, D. Kang. Abstain-R1: Calibrated abstention and post-refusal clarification via verifiable RL. *arXiv:2604.17073*, 2026.

[12] R. Shao et al. Spurious rewards: Rethinking training signals in RLVR. *arXiv:2506.10947*, 2025.

[13] Y. Yue et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *NeurIPS*, 2025.

[14] Z. Liu et al. Understanding R1-Zero-like training: A critical perspective. *arXiv:2503.20783*, 2025.

[15] Q. Yu et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv:2503.14476*, 2025.